Crossed uniqueness and the concise iPRA frontier

A uniform arithmetic rule and a measured 502-byte refinement

Prepared for Vladimir Reshetnikov

Research report with reproducible artifacts

12 November 2026

Abstract

On computationally normalized scalar-iterator terms, an arbitrary-seed crossed uniqueness rule derives all equality transport as well as equation induction. With the native case logic it gives a six-rule presentation bi-interpretable with full first-order iPRA. The new elementary argument derives symmetry, congruence and transitivity before induction. The subsequent scalar uniqueness-to-induction bridge is inherited and explicitly attributed.

Separately, a composition of existing implementation refinements reduces the smallest audited compact byte checker from 506 to 502 normalized bytes. This program has 269 significant tokens and 301 AST nodes. It checks the existing guarded presentation, not the new uniqueness calculus. Its strict input-validating companion has 1172 bytes.

The requested improvement below the complete current complexity envelope has *not* been established. The prior envelope is 253 tokens, 275 AST nodes and 506 bytes, attained by different programs. The new mathematical checker is larger. Neither absolute minimality nor a new Lean or Rocq formalization is claimed.

Contents

1 Result, comparison target, and claim boundaries **3**

1.1 Input contracts 3

2 A complete six-rule presentation **4**

2.1 Syntax and computation 4

2.2 All primitive rules 4

3 Equality derived before induction **5**

3.1 Numerical cases without transport 5

3.2 Symmetry 5

3.3 Congruence in any numerical template 6

3.4 Joining and transitivity 6

3.5 Full formula transport 6

4 The full logical and arithmetic bridge **7**

4.1 All intuitionistic operations 7

4.2 Scalar uniqueness and the noncircular induction bootstrap 8

5 Equiconsistency and both interpretation composites 8

6 The separate 502-byte refinement 9
6.1 Eager selection and delayed evaluation 10
6.2 The shorter mode selector 10

7 Executable evidence and reproduction 11

8 Limits and further research questions 11

A Attributed scalar uniqueness bootstrap 13

B An induction-free scalar presentation 13
B.1 The precise common background 13

C Zero-equation induction from uniqueness 14
C.1 Retraction rigidity 14
C.2 A numerical split suffices for induction 15
C.3 The unconditional zero-equation case 15

D Scalar iteration uniqueness already defines predecessor 15
D.1 Elementary identities by uniqueness 16
D.2 Two finite clocks 16
D.3 The modified ramp and its fusion law 16
D.4 The scalar comparison recurrence 17
D.5 Saturated iteration and predecessor 18
D.6 The exact dependency boundary 18

E Actual equality from scalar iteration uniqueness 19
E.1 A diagonal descent principle for numerical zero tests 19
E.2 Two Boolean comparison identities 20
E.3 An explicit scalar accumulator 20
E.4 Commutativity of the arithmetic maximum expression 21
E.5 Reflection and recovery of equation induction 22

F Equivalence, bi-interpretation, and the native calculus 22
F.1 The two proof transformations 22
F.2 What is inherited in the comparison with iPRA 23
F.3 Replacing progress in the native presentation 23
F.4 Scope of the result 24

1 Result, comparison target, and claim boundaries

The source snapshot is ProveIt commit

`6ffc49f55d355b548f7a8e1530570aee5de086b3`.

The overview alone does not identify the actual comparison target. All three incoming archives were inspected. A frozen source census contains 1380 distinct Python Git blobs at 5825 paths; all 90 current Research package tree identities match it. Its measurements and the classification of 106 smaller utility modules were reproduced.

The incoming `ipra_smaller_envelope.zip` supplies the token and AST records. The source `deterministic_introduction/implementation/code/compact_optional.py` supplies the byte record, Git blob `a0af7afb236b530eb30c63c5ab788151024271b7`. This 506-byte incumbent was not identified as the byte record in the overview table. The full census and comparison sources are included.

Complete module	Tokens	AST nodes	Bytes
Incoming token profile	**253**	280	599
Incoming AST profile	257	**275**	607
Audited byte control	267	303	**506**
New compact composition	269	301	**502**
Strict validating control	462	524	1183
New strict companion	463	519	1172
New uniqueness reference checker	1909	2087	5618

The census convention uses CPython 3.12.14: remove comments and genuine module/function/class docstrings, normalize with `ast.unparse` plus one final newline, count significant tokens and `ast.walk` nodes, and count UTF-8 bytes. The archive also supplies an actual normalized executable of the new reference checker. Its documented source, with docstrings retained under literal `ast.unparse`, measures 1914 tokens, 2097 AST nodes and 6686 bytes. No trusted code is omitted.

The envelope $(253, 275, 506)$ is empirical, not a lower-bound theorem for all possible axiomatizations, and it is not the size of one program. The compact composition lowers only the byte coordinate to 502, a four-byte or approximately 0.79% improvement.

1.1 Input contracts

The compact modules assume a finite integer/list certificate and a canonical closed guarded Eq/D public goal. They do not themselves validate that entire input promise. Their strict companions count alphabet and public-grammar validation within the complete module. The uniqueness checker has a different explicit syntax, validates open sequents as well as closed theorems, and counts normalization, substitution, scope checks, all rules and JSON input/output.

The mathematical contribution is the fusion of equality with uniqueness through an arbitrary initial equation and a crossed conclusion. Uniqueness as an alternative to induction is classical [6, 7]; the scalar completion is already in ProveIt [4]. The byte result composes three already proposed refinements: delayed elimination, eager tuple selection, and the shorter mode selector. It implements the existing calculus, not the new mathematical basis.

No complete implementation of the new basis below the current envelope is established. In particular, six rule names do not beat existing three-family clause calculi by themselves. The report supplies a mathematical primitive-basis proposal and a smaller byte checker, while leaving the requested full complexity breakthrough open.

2 A complete six-rule presentation

2.1 Syntax and computation

There is one numerical sort with raw terms

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in h is bound. Updates are fixed finite numerical templates with passive parameters; there are no function variables or universal evaluators for primitive-recursive descriptions.

Represent terms in full normal form for

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u]. \tag{Comp}$$

Every substitution is capture avoiding and followed by normalization. For $C(v, a, b) := I(v, a; u.b)$ with unused u,

$$C(0, a, b) \equiv a, \quad C(Sv, a, b) \equiv b, \quad I(1, a; u.h) \equiv h[a/u].$$

Definitional equality here uses only the two computation equations, not arbitrary proved arithmetic equalities.

Normalization is counted as part of the trusted specification. The repository supplies totality, canonical forms and

$$\mathrm{nf}(\mathrm{nf}(t)[\sigma]) = \mathrm{nf}(t[\sigma]). \tag{NCoh}$$

Its normalization-by-evaluation proof interprets a structurally smaller body as a total operation on normal terms, traverses the finite successor prefix of the normalized counter, and retains an iterator at a neutral counter. Environment laws establish substitution coherence. The reference checker uses equivalent direct reductions with capture avoidance and memoization [5, 1].

Formulas and their ordinary readings are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A,$$

$$\mathsf{Q}_y B := B(0) \to \forall y\, B(y), \qquad \mathsf{D}_{x,y} B := \exists x \big(B(x, 0) \to \forall y\, B(x, y)\big). \tag{Read}$$

D binds distinct x, y. Contexts are finite reusable lists of formulas. The native contradiction is $0 = S0$. The readings do not silently insert ordinary equality logic.

2.2 All primitive rules

All displayed expressions are normalized. The six families are

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \text{Hyp} \qquad \frac{}{\Gamma \vdash t = t}\ \text{Ref},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{QE},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \mathsf{Q}_y B(b,y) \vdash A}{\Gamma \vdash A}\ \mathrm{DE},$$

where b is fresh for Γ, A and the passive parameters of B;

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathrm{DI_c},$$

where z is fresh for Γ, t and the passive parameters of B; and

$$\boxed{\frac{\Gamma \vdash f(0) = a \qquad \Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash I(n,a;u.h(u)) = f(n)}\ \mathsf{U}.} \tag{U}$$

In U, $f(z), h(u)$ are fixed finite numerical templates. The eigenparameter z is fresh for Γ, a, n, h and all passive parameters of f. Thus h has no active-counter argument. The recurrence premise has no induction hypothesis. DI_c always uses cases, even at equation matrices.

There are no other logical axioms, computation axioms, transport rules or induction rules. Computation is exactly (Comp) in the syntax.

Lemma 2.1 (Structural admissibility)**.** *Weakening, renaming, ordinary-assumption cut and numerical substitution followed by normalization are admissible.*

Proof. Induct on finite derivations, freshening eigenparameters first. Normalization coherence identifies substituted premises and conclusions. Neither DI_c nor U has a mode that can change under substitution. Replace each use of a cut assumption by its proof. These are metatheoretic inductions on finite syntax, not numerical induction. □

3 Equality derived before induction

3.1 Numerical cases without transport

Lemma 3.1. *The case logic, without U, derives*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(n)} \quad \textit{and} \quad \frac{\Gamma, A(0) \vdash A(Sz)}{\Gamma \vdash \mathsf{Q}_y A(y)}, \tag{Cases}$$

with fresh z.

Proof. DI_c with an unused coordinate w introduces $\mathsf{D}_{w,y}A(y)$. For the first rule weaken the successor proof by $A(0)$. DE supplies $\mathsf{Q}_y A(y)$, and QE with the base gives $A(n)$, independent of the opened witness. For the second rule DE exports the opened Q assumption itself by Hyp. □

3.2 Symmetry

Given $s = t$, choose

$$f(z) = s, \qquad a = t, \qquad h(u) = u, \qquad n = 0$$

in U. Its recurrence is Ref. Its conclusion normalizes to $t = s$.

3.3 Congruence in any numerical template

For a template $H(u)$ and given $s = t$, choose

$$f(z) = I(z, s; u.H(u)), \qquad a = t, \qquad h = H, \qquad n = 1.$$

The base is $s = t$, while both sides of the recurrence normalize to $H(I(z, s; H))$. Ref proves it. The conclusion is $H(t) = H(s)$; symmetry gives ordinary congruence. Templates may duplicate or omit the argument, or place it beneath additional iterator binders.

3.4 Joining and transitivity

Lemma 3.2 (Common-source joining)**.** *From $s = t$ and $s = u$ in one retained context, derive $t = u$ without numerical induction.*

Proof. Let $f(z) = C(z, s, u)$ and use the identity update. Its recurrence is

$$u = C(z, s, u).$$

At zero this is $u = s$, by symmetry of $s = u$; at a successor it is $u = u$, by Ref. Numerical cases prove the recurrence. U with $a = t, n = 1$ has base $s = t$ and conclusion $t = u$. □

From $s = t, t = u$, first derive $t = s$ and join at t to get $s = u$. Thus transitivity is derived. From $s = t, H(s) = r$, congruence supplies $H(s) = H(t)$ and joining gives $r = H(t)$, the repository's one-term crossed transport rule.

3.5 Full formula transport

Theorem 3.3 (Leibniz transport)**.** *For every native formula template $F(v)$,*

$$\Gamma \vdash s = t, \quad \Gamma \vdash F(s) \quad \Longrightarrow \quad \Gamma \vdash F(t).$$

Proof. Induct on F. At an equation use congruence on both sides, symmetry and transitivity.

For $F(v) = \mathsf{Q}_y B(v, y)$, temporarily assume $B(t, 0)$. Reverse matrix transport gives $B(s, 0)$; QE at Sz gives $B(s, Sz)$; forward matrix transport gives $B(t, Sz)$. Conditional Q introduction (Cases) finishes.

For $F(v) = \mathsf{D}_{x,y} B(v, x, y)$, open at fresh b and retain $\mathsf{Q}_y B(s, b, y)$. Introduce the desired D formula at witness b. From its base $B(t, b, 0)$, reverse matrix transport gives $B(s, b, 0)$, QE gives $B(s, b, Sz)$, and forward transport gives $B(t, b, Sz)$. DI_c closes the desired formula and DE discharges b. Each binder case uses only the strictly smaller matrix. □

This elementary fusion theorem assumes neither predecessor, numerical equality tests, successor injectivity nor an induction-hypothesis rule.

4 The full logical and arithmetic bridge

4.1 All intuitionistic operations

The intended comparison is with full first-order iPRA. The following macros are the inherited induction-independent case-logic construction [2]; they now apply because equality was already recovered.

Replace Q inside macros by its interderivable D form with unused witness. On Q-free formulas define

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB.$$

With capture avoidance, computation, Ref and case introduction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{w,y}G_yA.$$

For reverse padding, open D, supply its true zero row, and specialize at 1.

The selector between formulas is

$$M_v(s = t, r = q) := (C(v, s, r) = C(v, t, q))$$

and, with aligned fresh binders,

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Maximum input binder depth decreases. Padding and binder congruence yield $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$; $p(1) = 1$ and $p(2) = 0$ are computations. With fresh coordinates and unused w, set

$$\begin{aligned} \bot_* &:= (0 = 1),\\ A \Rightarrow_* B &:= \mathsf{D}_{w,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{w,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* v\, M_v(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{w,z}G_zA(z). \end{aligned} \tag{Logic}$$

Implication introduction is DI_c using its zero row as the assumption; elimination opens and specializes at 1. Existential introduction supplies the witness, and elimination opens it freshly and removes the gate. Disjunction injections use 0 and 1; elimination uses numerical cases on the selector and the two retained branch implications. Conjunction introduction uses selector cases and elimination uses rows 1 and 2. The universal keeps zero as a conjunct and supplies successors in the other conjunct; coverage yields every instance. Its introduction uses zero and successor substitutions of a proof at a fresh parameter.

Coverage $(n = 0) \vee_* \exists_* m(n = Sm)$ follows by cases and Ref. From $0 = 1$, congruence in $v \mapsto C(v, s, t)$ gives every equation; formula-structural induction gives explosion. These supply all ordinary intuitionistic rules with the usual eigenvariable conditions. No excluded middle for arbitrary formulas is added. The ordinary realization has the explicit falsity bridge $(0 = S0) \to \bot$.

4.2 Scalar uniqueness and the noncircular induction bootstrap

In U choose $a = f(0)$, use Ref for the base and reverse its conclusion:

$$\frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash f(n) = I(n, f(0); u.h(u))}. \tag{UR}$$

This is exactly the existing scalar uniqueness principle [4]. Its equality and logical background has already been recovered.

Appendix A gives the complete inherited proof that UR yields equation induction. Its dependency order is important:

1. UR gives $I(n, 0; u.Su) = n$ and retraction rigidity. If $e(0) = 0$, $e(e(x)) = e(x)$ and $e(S(e(x))) = S(e(x))$, take $h(u) = e(S(e(u)))$, $r(n) = I(n, 0; h)$. Common-update uniqueness gives $e(r(n)) = r(n)$; then r has successor update, so $r(n) = n$ and $e(n) = n$. For $q(n) = 0$, coverage and separation decide the property and the term $e(n) = C(q(n), n, 0)$ provides the retraction split. This derives contextual induction for zero equations.

2. A scalar predecessor is constructed from finite clocks, a ramp, a comparison iterator and bounded growth. Common-update uniqueness proves $P0 = 0$ and $P(Sn) = n$, without using pairing, a general recursion compiler or equation induction.

3. Define $d(a, b) = I(b, a; u.Pu)$ and $\mathrm{sg}(a) = C(a, 0, 1)$. A scalar diagonal-descent argument and the explicit accumulator

$$\mu(n, a) = d(n, d(n, a)), \quad \phi(n, a, b) = \mu(n, a) + \mu(d(n, a), d(b, a))$$

prove $a + d(b, a) = b + d(a, b)$. Thus

$$\Delta(a, b) = \mathrm{sg}(d(a, b) + d(b, a))$$

is Boolean and $\Delta(a, b) = 0 \leftrightarrow a = b$ on the entire domain.

4. Translate an equation-induction base and step for $s(n) = t(n)$ to $\Delta(s(n), t(n)) = 0$, use zero-equation induction, and reflect back. The retained context may contain arbitrary first-order formulas.

The substantial scalar construction is inherited and attributed, not claimed as a new result of U. Its full equations and proofs appear in the appendix. Once equation induction is recovered, from $\Gamma, E(z) \vdash E(Sz)$ work under $E(0)$ to derive $E(Sw)$ and use conditional Q introduction. This recovers the old equation-progress rule.

5 Equiconsistency and both interpretation composites

Theorem 5.1 (Exact native equivalence)**.** *For every finite sequent of the repository's native calculus $\mathcal{N}$,*

$$\Gamma \vdash_{\mathcal{N}} A \quad \Longleftrightarrow \quad \mathrm{nf}(\Gamma) \vdash_{\mathcal{U}} \mathrm{nf}(A).$$

Proof. Translate native rules to normal forms. Computation becomes Ref, transport is Section 3, progress is Section 4, and the other rules are unchanged.

Conversely, the native calculus proves normalization equalities and derives U by equation induction on $I(z, a; h) = f(z)$. Its base uses $f(0) = a$ and symmetry. Its successor step uses the

induction equation, congruence under h, computation and the retained recurrence $f(Sz) = h(f(z))$. All the remaining U-calculus rules are native rules or derived reflexivity. □

Theorem 5.2 (Full first-order iPRA)**.** *The ordinary realization of $\mathcal{U}$ is equiconsistent and bi-interpretable with full first-order iPRA. Both numerical domains are whole, equality is actual, and the two numerical composite isomorphisms have graph $x = y$.*

Proof. Compose the exact native equivalence with the scalar bridge [3]. In iPRA interpret each fixed iterator by primitive recursion, and Q,D by (Read). Equation induction validates U.

Conversely, predecessor and equality reflection supply successor injectivity and characteristics for every fixed quantifier-free formula. Finite truth tables recover quantifier-free induction. The inherited scalar construction supplies pairing π and projections L, H. For a fixed primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

use

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big),$$
$$f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves the defining equations. This is an external compiler for fixed descriptions, not an object-language uniform evaluator.

The domain formula is $x = x$ in each direction and equality is unchanged. Every double-translated operation satisfies its original defining recurrence. Internal uniqueness identifies the two operations, making $x = y$ internally total, bijective and operation preserving.

For formulas the logical macros have their ordinary round trips. The essential paired composite is

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big).$$

Forwards, open D, assume the base, use QE at a fresh parameter and introduce the macros. Backwards, eliminate the existential and use the implication and universal instances in DI_c. The Q composite is its conditional universal. Structural induction supplies both composites for all formulas, including quantified ones. The explicit falsity bridge preserves contradiction in both directions, giving equiconsistency. □

This is a written proof using the stated inherited scalar theorems. It is stronger than mutual interpretability or agreement on standard numerals, but is not a new proof-assistant verification.

6 The separate 502-byte refinement

This complete program checks the existing guarded public calculus, not $\mathcal{U}$:

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d
     else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
```

```
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a :=
     i(*a, t, 2))
    c(i((a, i(t, s))[(m, a) == (s and [s, u] or [t] * 3, i(t, u))], [(k := (~-k))]), g + [i(h, (m
    or 3 % len(a)) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

The JSON list is consumed from the end: the first item popped is the goal, followed by four-field instructions. Acceptance is normal exit and exactly `ACCEPT` followed by a newline. Exceptions reject; a well-typed unused prefix is allowed.

6.1 Eager selection and delayed evaluation

At the old scope floor, let C_0 denote the paired elimination test and $R = i(t, s)$. The old selection C_0 and R or a becomes $(a, R)[C_0]$. This Python rewrite is not valid in general. Here the pure opener i preserves a list's root length and sends an integer at root depth zero either to itself or to zero independently of the image. When both openings terminate, false R implies $i(t, u) = R$. If C_0 is true, $a = i(t, u) = R$; therefore the old fallback and new selection agree even then. If the test is false both select a.

Extra eager evaluation can reject invalid unused fields, but the eager-only transformation adds no successful integer/list trace. Canonical completeness survives because intended source terms are well scoped and unused fields can be standardized to zero.

The delayed placement is inherited from the 506-byte control. For a nonzero major, construction of the retained formula does not depend on the goal. The recursive major check does not mutate its caller's local scope or context. For a false major, the reconstructed elimination major is nonempty, so the test is false both before and after introduction. The final expression computes the selected goal at the old floor, decrements the floor, and only then performs successor opening and extends the immutable context.

6.2 The shorter mode selector

The selection changes from

$$(m \text{ or } \sim \text{len}(a) \bmod 2) \text{ and } k \quad \text{to} \quad (m \text{ or } 3 \bmod \text{len}(a)) \text{ and } k.$$

At meaningful formula root lengths 1,2,3 both choose $0, k, 0$ for a false major; a true major selects k in both.

For raw aliases, truthiness differs only at odd surplus lengths at least 5. A surplus formula root decodes as administrative truth. If the branch left the goal unchanged, that call already has a true goal. If introduction assigned the opened matrix, successful unpacking forces the entry goal to be an empty truth alias or a D formula with a surplus-root matrix. The latter is $\mathsf{D}(\text{truth})$, provable with witness zero. Changed local hypotheses therefore cannot justify a false entry goal, and they do not escape their call. Empty-root division errors only remove raw certificates, not canonical proofs.

This is semantic soundness and preservation of canonical proof compilation, not equality of the entire raw certificate languages. The strict companion counts exact integer/list and public-goal validation within its 1172 bytes. The detailed argument, intermediate controls and provenance are in `implementation/compact/`.

7 Executable evidence and reproduction

The strict uniqueness checker accepts exactly the record keys `scope`, `context`, `goal`, `proof`. A closed theorem has scope 0 and context `[]`. Terms are natural-number de Bruijn indices or `["z"]`, `["s",t]`, `["i",n,a,h]`; the last child binds index 0. Formulas are `["=",s,t]`, `["q",B]`, `["d",B]`. D binds y at 0 and x at 1.

The six tags are `hyp`, `refl`, `qe`, `de`, `di`, `u`. U is encoded as

```
["u", f, a, h, n, base_proof, step_proof]
```

Both templates have one bound variable. The checker introduces the recurrence eigenvariable itself and raises passive indices, preventing hidden counter dependence. All scopes are checked before normalization, so even ill-scoped subterms erased by computation reject.

Numerical cases in generated certificates expand into DI_c, DE, QE and Hyp. There is no hidden seventh primitive. Eight positive certificates include symmetry, nontrivial template congruence, common-source joining using both assumptions, transitivity, and nondefinitional identity and successor iterator laws. Eleven targeted invalid cases reject wrong recurrence and base premises, wrong closure bases, witness escape, unbound variables, Boolean index aliases and malformed instruction lengths. A separate same-run review found no concrete defect in the binder handling.

The compact reproduction ran 267 checker processes, with ordinary and optimized Python, 21 inherited positive/adversarial fixtures, strict malformed-prefix checks and an intentionally old-only eager certificate. There were zero unexpected outcomes. These executions validate concrete implementation behavior; they do not formalize the universal theorems.

From the archive root:

```
python3 reproduce.py
python3 implementation/uniqueness/uniqueness_checker.py \
  < implementation/uniqueness/certificates/common_source_join.json
python3 implementation/compact/strict1172.py \
  < implementation/compact/fixtures/computation_zero_local.json
python3 reference/research_census/reproduce_audit.py
python3 build.py
python3 verify_manifest.py
```

The census parses frozen sources without executing them. Checkers and validation use only the Python standard library; the PDF build needs LaTeX. Finite recursion or memory limits may reject large valid proofs. Mathematical completeness uses unbounded-resource semantics; resource exhaustion is never acceptance.

8 Limits and further research questions

The new six-rule basis has a larger checker. The 502-byte program improves one coordinate for the inherited basis. These two results must not be combined into a claim that the new mathematical basis achieves that byte record.

1. Can seeded crossed uniqueness receive a compact codec that retains all freshness and counter-independence checks? One complete program below all current coordinates would meet the strongest form of the request.

2. Can a clause calculus fuse U with witness discharge without complicating the equality derivation? Existing three-family calculi already set a demanding comparison.
3. Can the scalar uniqueness bootstrap and this composition be formalized in Lean? The short elementary equality theorem is a useful first target.
4. Can normalization share the arity traversal while respecting sorted positions, protected dummy slots and reindexing? A naive rewrite of all ternary nodes is invalid.
5. Can a changed-domain relational bi-interpretation reduce the entire checker? Fixed-language term-definability obstructions do not rule it out; both definable composite isomorphisms still require proofs.

A smaller checker gives an upper bound on attainable size. An absolute lower bound requires a fixed implementation language, input contract, normalization convention and admissible interpretation class.

A Attributed scalar uniqueness bootstrap

The following sections reproduce the existing scalar uniqueness completion from the pinned ProveIt sources under their supplied license. They make the inherited implication used in Section 4 directly reviewable. Claims of novelty inside the reproduced text refer to that earlier report. The new elementary equality argument of this report is Section 3.

B An induction-free scalar presentation

The new arithmetic principle is unconditional uniqueness of scalar iteration. The general idea of replacing induction by recursion uniqueness is classical. The result proved here is more specific: uniqueness for a *single scalar state with no counter argument in the update* already suffices, in the original scalar language. It closes the explicit equality-reflection gap in the incoming `scope_kernel` report. It does not rely on a general primitive recursion compiler or on numeric pairing before induction has been recovered.

B.1 The precise common background

Use one numerical sort and fixed finite term syntax

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

The binder binds u only in h. Equivalently, each finite earlier-stage body $h(u, \vec{p})$ indexes one first-order function symbol $I_h(n, a, \vec{p})$. Bodies are not function variables, and the theory postulates no universal evaluator for primitive-recursive descriptions. All substitutions avoid capture.

The surrounding logic is intuitionistic first-order logic with actual equality and genuine falsity $\bot$. Its usual logical constructors are $\wedge, \vee, \to, \forall, \exists, \bot$; negation abbreviates implication to $\bot$. In particular, no double-negation elimination or excluded middle for arbitrary formulas is assumed.

The nonlogical axioms are the universally closed instances

$$\begin{aligned} I(0, a; u.h) &= a, \\ I(Sn, a; u.h) &= h[I(n, a; u.h)/u], \\ \forall n\,(n = 0 \vee \exists m\, n = Sm), & \\ 0 = S0 &\longrightarrow \bot. \end{aligned}$$

No successor injectivity axiom, equality characteristic, primitive pairing, or induction axiom occurs in this background.

The uniqueness theory T_{UR} adds the single inference schema

$$\boxed{\frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash f(t) = I(t, f(0); u.h(u))}.} \tag{UR}$$

Here $f(z)$ and $h(u)$ are fixed finite term templates. The update has no free z. The variable z is fresh for the retained context, the target t, and all passive parameters. Crucially, the premise is proved in Γ itself: there is no additional induction hypothesis. The seed $f(0)$ is read from the same template, so there is no separate base premise. This is an inference schema over finite syntax, not quantification over a sort of functions.

Let T_{eq} use the same background and replace UR by contextual induction on each literal equation $s(n) = r(n)$:

$$\frac{\Gamma \vdash s(0) = r(0) \qquad \Gamma, s(z) = r(z) \vdash s(Sz) = r(Sz)}{\Gamma \vdash s(t) = r(t)}, \tag{EI}$$

with the same usual freshness condition.

Theorem B.1 (Scalar uniqueness completion)**.** *For every first-order sequent over the scalar language,*

$$\Gamma \vdash_{T_{\mathrm{UR}}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{T_{\mathrm{eq}}} A.$$

Consequently T_{UR} is bi-interpretable and equiconsistent with full first-order iPRA. The arithmetic comparison uses the entire numerical domain and actual equality; both round trips have identity graph $x = y$.

The hard implication is proved below. The order of the proof prevents the usual circularity: UR first gives zero-equation induction and a predecessor, then a scalar accumulator gives equality reflection, and only then is arbitrary equation induction recovered. General primitive recursion is compiled after that point.

C Zero-equation induction from uniqueness

The argument in this section is the retraction argument already present in the incoming `scope_kernel` report. It is reproduced to make the dependency on equality reflection exact. It uses neither predecessor nor a general numerical equality characteristic.

All calculations are under one fixed retained first-order context Γ. Fresh displayed variables are absent from Γ and all passive parameters. UR first gives

$$I(n, 0; u.Su) = n. \tag{Id}$$

Use the template $f(z) = z$ and update $h(u) = Su$; its recurrence premise is reflexivity. Common-update uniqueness, obtained by applying UR separately to two terms and replacing equal seeds, is used below.

C.1 Retraction rigidity

Let $e(x)$ be a fixed scalar term template. Suppose Γ proves

$$e(0) = 0, \qquad e(e(x)) = e(x), \qquad e(S(e(x))) = S(e(x)). \tag{Ret}$$

Then Γ proves $e(t) = t$ for every target t.

To see this, put $h(u) = e(S(e(u)))$ and $r(n) = I(n, 0; u.h(u))$. Idempotence in Ret gives $e(h(u)) = h(u)$ and $h(e(u)) = h(u)$. Therefore the template $f(n) = e(r(n))$ has initial value zero and unconditional recurrence

$$f(Sn) = e(h(r(n))) = h(r(n)) = h(e(r(n))) = h(f(n)).$$

UR gives $e(r(n)) = I(n, 0; u.h(u)) = r(n)$. The last equation of Ret now yields

$$r(Sn) = h(r(n)) = S(e(r(n))) = S(r(n)), \qquad r(0) = 0.$$

A second application of UR, followed by Id, gives $r(n) = n$. Replacing $r(n)$ by n in the previous equality proves $e(n) = n$. No induction was used, and no claim that the standard finite successor orbit exhausts a model was used.

C.2 A numerical split suffices for induction

For a fixed formula $A(n)$, suppose

$$\Gamma \vdash A(0), \qquad \Gamma, A(z) \vdash A(Sz).$$

Suppose moreover that $A(n)$ is decidable under Γ and that a term $e(n)$ satisfies $e(n) = n$ under $A(n)$ and $e(n) = 0$ under $\neg A(n)$. Cases give $A(e(n))$: in the first case use $A(n)$, and in the second use the proved base $A(0)$. Applying the true-case identity at $e(n)$ gives $e(e(n)) = e(n)$. The induction-step derivation, instantiated at $e(n)$, gives $A(S(e(n)))$, hence $e(S(e(n))) = S(e(n))$. The base similarly gives $e(0) = 0$. Retraction rigidity proves $e(n) = n$; transporting $A(e(n))$ gives $A(n)$.

This is a metatheorem about a justified numerical split. It supplies no decision procedure, comprehension principle, or induction schema for arbitrary quantified formulas.

C.3 The unconditional zero-equation case

Write $1 = S0$ and $C(v, a, b) = I(v, a; u.b)$. The two computations give $C(0, a, b) = a$ and $C(Sv, a, b) = b$. The separation axiom $0 = 1 \to \bot$ implies $Sv \neq 0$: otherwise applying $w \mapsto C(w, 0, 1)$ gives $1 = 0$, hence a contradiction. Coverage therefore decides $q(n) = 0$ for every fixed scalar term $q(n)$. The term

$$e(n) = C(q(n), n, 0)$$

has exactly the split required in the preceding subsection. Consequently the following contextual rule is derived:

$$\frac{\Gamma \vdash q(0) = 0 \qquad \Gamma, q(z) = 0 \vdash q(Sz) = 0}{\Gamma \vdash q(t) = 0}. \tag{ZI}$$

Every substitution and case distinction is intuitionistic. The retained context can contain arbitrary quantified formulas, provided the fresh induction variable is absent from it.

D Scalar iteration uniqueness already defines predecessor

This section gives the predecessor portion of the completion of the scalar-uniqueness direction identified in the incoming `scope_kernel` report. Its new ingredient is a modified scalar ramp whose invariant is a direct fusion law. Equality reflection is proved independently in Section E.

Work in ordinary intuitionistic first-order logic with actual equality. Terms are generated by variables, 0, S, and $I(n, a; u.h)$, with only u bound in h. Assume the two iterator equations,

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

and numerical coverage $n = 0 \vee \exists m\, n = Sm$. Replace equation induction by the contextual scalar uniqueness rule

$$\frac{\Gamma \vdash f(Sz) = h[f(z)/u]}{\Gamma \vdash f(t) = I(t, f(0); u.h)} \quad \text{(UR)},$$

where z is fresh for Γ and the passive parameters, $f(z)$ is a term template, and $h(u)$ has no free z. All bodies are finite earlier syntax. A useful derived rule says that f and g are equal whenever they have provably equal initial values and the same unconditional autonomous update. Apply UR to both and transport the initial value. No induction hypothesis is permitted in the recurrence premise.

Theorem D.1. *The preceding theory defines a finite scalar term P and proves $P(0) = 0$ and $P(Sn) = n$. Consequently it proves successor injectivity. The derivation does not use separation of 0 and $S0$.*

D.1 Elementary identities by uniqueness

Write $1 = S0$, set $C(n, a, b) = I(n, a; u.b)$, $Z(n) = C(n, 1, 0)$, and $a + b = I(b, a; u.Su)$. Coverage gives $Z(n) = 0 \vee Z(n) = 1$ and

$$C(0, a, b) = a, \quad C(Sn, a, b) = b, \quad C(e, Sa, Sb) = S(C(e, a, b)).$$

The last identity follows directly by cases on e. All commutative-monoid laws for addition follow from common-update uniqueness. In detail, n has initial value 0 and update S, so $0 + n = n$; $S(a+n)$ has initial value Sa and update S, so $Sa+n = S(a+n)$. Thus $n+a$ has initial value a and update S, proving $n + a = a + n$. Finally $a + (b + n)$ has initial value $a + b$ and update S, proving associativity. These facts justify the usual finite sum notation $2a = a + a$ and $3a = a + a + a$.

For any fixed body h, the following are also common-update uniqueness proofs:

$$I(m + n, a; h) = I(n, I(m, a; h); h), \qquad I(Sn, a; h) = I(n, h(a); h).$$

For the first compare the two sides as functions of n, with update h and initial value $I(m, a; h)$; for the second the initial value is $h(a)$. A constant function fixed by h is likewise the corresponding iterator.

D.2 Two finite clocks

Define

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

Coverage on the counter of N proves that $N(n)$ is 0 or 1. Hence $Z(Z(N(n))) = N(n)$ and $N(n) + Z(N(n)) = 1$ by two finite cases. The computation rule gives $N(n + 2) = N(n)$. The function $N(2n)$ has initial value 0 and identity update, whereas $N(3n)$ has initial value 0 and update Z. UR therefore gives

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n).$$

The range of j lies in $\{0, 1, 2\}$: at $u = 0$ its value is 1; at $u = Sv$ the term $Z(u)$ is zero and $2N(u)$ is 0 or 2. Coverage on the counter of M therefore proves that its range also lies in $\{0, 1, 2\}$. Direct computation gives

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

Thus $M(n + 3) = M(n)$ by three computations and the three cases for $M(n)$. UR with the identity update gives $M(3n) = 0$, and one further computation gives $M(3n + 1) = 1$.

D.3 The modified ramp and its fusion law

Use the update

$$g(u) = u + N(M(u)) + 1, \qquad G(n) = I(n, 0; u.g(u)).$$

It differs from the older ramp update $u + M(u) + 1$ away from its intended orbit. That difference makes the following law valid at every argument:

$$M(g(u)) = Z(M(u)). \tag{1}$$

Indeed, when $M(u) = 0$, $g(u) = u + 1$ and the new M value is 1; when $M(u) = 1$, $g(u) = u + 2$ and the new value is 0; when $M(u) = 2$, $g(u) = u + 1$ and the new value is again 0. Consequently $M(G(n))$ has initial value 0 and update Z. Uniqueness, without any induction hypothesis, proves

$$M(G(n)) = N(n). \tag{2}$$

As N fixes the two Boolean values, this gives

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3, \qquad G(n + 3) = G(n) + N(n) + 4. \tag{3}$$

The last two equations follow by two or three computations and the two cases for $N(n)$.

The functions $G(2n)$ and $3n$ both start at zero and have update $v \mapsto v + 3$, so UR yields $G(2n) = 3n$. For the less immediate scaling equation, put

$$f(n) = G(3n), \qquad k(n) = 3G(n) + N(n), \qquad H_0(v) = v + M(v) + 4.$$

Equation (2) and parity scaling give $M(f(n)) = N(n)$. The identities $M(3a) = 0$ and $M(3a+1) = 1$, together with the two cases for $N(n)$, give $M(k(n)) = N(n)$. Equations (3) show $f(Sn) = H_0(f(n))$. In those same two Boolean cases,

$$\begin{aligned} k(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)) \\ &= 3G(n) + 2N(n) + 4 = H_0(k(n)). \end{aligned}$$

Both start at zero. A second common-update uniqueness proof therefore gives

$$G(2n) = 3n, \qquad G(3n) = 3G(n) + N(n). \tag{4}$$

D.4 The scalar comparison recurrence

Set $H(u) = C(N(u), G(u), u)$. From (4) and the parity identities,

$$H(2u) = 3u, \quad H(1) = 1, \quad H(3u) = 3H(u).$$

For the last identity split $N(u) = 0$ or 1; in the zero case use $G(3u) = 3G(u)$, and in the one case both sides retain the corresponding odd argument. No division theorem is used. The functions $I(y, 3a; H)$ and $3I(y, a; H)$ have the same initial value $3a$ and update H, hence

$$I(y, 3a; H) = 3I(y, a; H), \qquad I(y, 1; H) = 1.$$

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N(I(y, E(x); H)).$$

The rotation identity and the displayed equations yield

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{5}$$

For the final equality, rotate the first iterator step at the initializer $2E(x)$, replace $H(2E(x))$ by $3E(x)$, move the factor three through the iterator, and use $N(3v) = N(v)$.

D.5 Saturated iteration and predecessor

Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

Equation (5) and selector compatibility with successor give

$$B_{Sm}(Su) = SB_m(u), \qquad B_{Sm}(0) = 1.$$

As functions of t, both $W(St, Sm)$ and $SW(t, m)$ start at one and have update B_{Sm}. Therefore UR proves

$$W(St, Sm) = SW(t, m). \tag{6}$$

The function $n \mapsto L(Sn, n)$ starts at zero and has identity update by (5), so it is identically zero by UR. By the already universal equation (6), $n \mapsto W(n, n)$ starts at zero and has update S, so it equals n. It follows that

$$W(Sn, n) = B_n(W(n, n)) = C(L(Sn, n), n, Sn) = n. \tag{7}$$

Finally put

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big).$$

The zero equation is immediate. For fixed m, the function

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big)$$

has the same initial value and update as $W(t, m)$, because (5) identifies its update pointwise with B_m. UR yields $V(t, m) = W(t, m)$. Using (7),

$$P(Sm) = V(Sm, m) = W(Sm, m) = m.$$

Applying P to $Sx = Sy$ now gives $x = y$.

D.6 The exact dependency boundary

Every appeal to induction in the predecessor bootstrap has been replaced by a specified unconditional common-update uniqueness instance. All case distinctions use coverage and finite ranges; no separation axiom, equality characteristic, or induction on an equation has been smuggled in. One also obtains $d(x, y) = I(y, x; u.Pu)$ and

$$d(Sx, Sy) = d(x, y), \qquad d(a + n, n) = a,$$

again by common updates P and the identity, respectively. Consequently addition is cancellative, and $m(x, y, z) = d(x + z, y)$ is a Mal'cev operation: $m(x, x, z) = z$ and $m(x, y, y) = x$.

The equality-reflection step needs a different argument. The older proof used

$$C\big(\mathrm{sg}(d(x, y)), x, y + d(x, y)\big) = x, \qquad \mathrm{sg}(v) = C(v, 0, 1).$$

Its familiar proof invokes the equality at y as an induction hypothesis. The left side has a counter-dependent reset recurrence, not an immediate autonomous scalar recurrence. It is not imported into this bootstrap. Instead, Section E constructs an explicit scalar accumulator using Boolean diagonal descent, proves a maximum commutativity identity, and obtains actual equality reflection from it.

E Actual equality from scalar iteration uniqueness

Work in intuitionistic first-order logic with actual equality, the scalar iterator computation equations, numerical zero/successor coverage, and $0 \neq S0$. Assume the contextual scalar uniqueness rule

$$\frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash f(t) = I(t, f(0); u.h(u))}, \tag{UR}$$

where z is fresh for Γ, t and all passive parameters. The update h has no active counter argument. Every displayed function in what follows is a fixed finite scalar term abbreviation; there are no function variables and no hidden product-state iteration.

The preceding scalar predecessor construction supplies a term P with $P0 = 0$ and $P(Sx) = x$. Addition, defined by $a + b = I(b, a; u.Su)$, satisfies the commutative-monoid equations by common-update uniqueness. Define

$$d(a,b) = I(b,a;u.Pu), \qquad Z(a) = C(a,1,0), \qquad \mathrm{sg}(a) = C(a,0,1), \qquad C(v,a,b) = I(v,a;u.b).$$

The following elementary equations use only UR and computation:

$$\begin{aligned} d(a,0) &= a, & d(a,Sb) &= P(d(a,b)), \\ d(Sa,Sb) &= d(a,b), & d(a,a) &= 0, & d(0,a) &= 0. \end{aligned} \tag{1}$$

For cancellation, the two terms $b \mapsto d(Sa, Sb)$ and $b \mapsto d(a, b)$ have common initial value a and common update P. The diagonal $a \mapsto d(a, a)$ has the identity update and initial value zero. The constant-zero function and $a \mapsto d(0, a)$ have common update P and initial value zero. Thus no equation-induction assumption is hidden here.

E.1 A diagonal descent principle for numerical zero tests

UR implies the following constant-recurrence principle:

$$\Gamma \vdash f(Sz) = f(z) \quad \Longrightarrow \quad \Gamma \vdash f(t) = f(0). \tag{Const}$$

Apply UR with identity update to f and to the constant function $f(0)$.

Fix a finite arity $r \geq 1$ and a scalar term $q(x_1, \ldots, x_r)$. Suppose that, with any passive parameters fixed,

$$q(Sx_1, \ldots, Sx_r) = q(x_1, \ldots, x_r)$$

and that q is zero whenever any of its displayed arguments is zero. Then

$$q(x_1, \ldots, x_r) = 0. \tag{DD}$$

Indeed, coverage in each argument gives $q(Px_1, \ldots, Px_r) = q(x_1, \ldots, x_r)$: if any argument is zero, both sides are zero; otherwise the successor identity applies. Now put

$$f(k) = q(d(x_1, k), \ldots, d(x_r, k)).$$

The defining equation for d gives $f(Sk) = f(k)$, so Const makes f constant. At $k = x_1$ its first argument is $d(x_1, x_1) = 0$; hence $f(x_1) = 0$. At $k = 0$ it is the desired $q(x_1, \ldots, x_r)$. This argument uses scalar common-update uniqueness. It does not quantify over tuples or invoke a simultaneous-recursion rule.

For Boolean-valued terms p, q, use

$$\mathrm{or}(p,q) = C(p,q,1), \qquad \mathrm{xor}(p,q) = C(p,q,Z(q)).$$

Here Boolean-valued means that the term is provably either 0 or 1. Finite cases show that $\mathrm{xor}(p,q) = 0$ if and only if $p = q$. This finite Boolean fact is not reflection of arbitrary numerical equality.

E.2 Two Boolean comparison identities

First, DD gives

$$Z(d(Sn,a)) = \mathrm{sg}(d(a,n)). \tag{B1}$$

To verify its DD premises, let $q(n,a)$ be the Boolean xor of the two displayed sides. Simultaneously replacing n, a by Sn, Sa cancels one successor in each subtraction, so $q(Sn,Sa) = q(n,a)$. If $a = 0$, both sides are zero. If $n = 0$, split a into zero and successor: the former case again gives two zeros, while $a = Sb$ gives $d(1,Sb) = d(0,b) = 0$ and $\mathrm{sg}(Sb) = 1$, so both sides are one. Thus both zero-coordinate boundaries vanish and DD applies.

Second, DD gives

$$Z\big(d(d(x,a),d(b,a))\big) = \mathrm{or}\big(Z(d(x,a)), Z(d(x,b))\big). \tag{B2}$$

Again use the Boolean xor $q(x,a,b)$ of the two sides. When all three arguments are successors, simultaneous cancellation reduces the expression to $q(x,a,b)$. The three zero-coordinate boundaries are as follows. If $x = 0$, both displayed sides are one. If $a = 0$, the left side is $Z(d(x,b))$, and the right side is $\mathrm{or}(Z(x), Z(d(x,b)))$; coverage on x makes them equal. If $b = 0$, the left side is $Z(d(x,a))$, and the right side is $\mathrm{or}(Z(d(x,a)), Z(x))$; coverage on x again proves equality. In both latter arguments, the $x = 0$ branch makes the other Boolean one, whereas the successor branch makes $Z(x) = 0$. Consequently DD proves the vanishing of the xor, and Boolean cases give the stated identity.

Coverage and the successor equation for addition also give

$$\mathrm{or}(\mathrm{sg}(u), \mathrm{sg}(v)) = \mathrm{sg}(u+v). \tag{B3}$$

For $u = 0$ this reduces to the sign of v; for successor u, both sides are one.

E.3 An explicit scalar accumulator

Define the two scalar terms

$$\begin{aligned} \mu(n,a) &= d(n,d(n,a)), \\ \phi(n,a,b) &= \mu(n,a) + \mu(d(n,a),d(b,a)). \end{aligned} \tag{2}$$

These are finite compositions of d and addition. In particular, ϕ is not introduced by a counter-dependent recursion. We now prove its useful counter-dependent equation from its explicit term.

Coverage on a and the monus equations give

$$P(d(Sn,a)) = d(n,a). \tag{Pd}$$

At $a = 0$ this is $P(Sn) = n$. At $a = Sa'$ it follows from $d(Sn,Sa') = d(n,a')$ and $P(d(n,a')) = d(n,Sa')$. Split the value $r' = d(Sn,a)$ into zero and successor. If $r' = 0$, then $d(n,a) = 0$ by Pd, so

$\mu(Sn, a) = Sn$ and $\mu(n, a) = n$. If $r' = Sr$, then $d(n, a) = r$ and $\mu(Sn, a) = d(Sn, Sr) = d(n, r) = \mu(n, a)$. In both cases,

$$\mu(Sn, a) = \mu(n, a) + Z(d(Sn, a)). \tag{M}$$

Put $h = d(b, a)$ and repeat the same split of $r' = d(Sn, a)$. If $r' = 0$, the first argument of the second μ term is zero at both n and Sn, and $\phi(Sn, a, b) = Sn$, $\phi(n, a, b) = n$. If $r' = Sr$, the first summand is unchanged, while M gives

$$\mu(Sr, h) = \mu(r, h) + Z(d(Sr, h)).$$

Associativity therefore proves, in both cases,

$$\phi(Sn, a, b) = \phi(n, a, b) + Z(d(d(Sn, a), d(b, a))).$$

Apply B2 at $x = Sn$, then B1 to each Boolean on its right, and finally B3. The resulting recursion specification is

$$\boxed{\phi(0, a, b) = 0, \qquad \phi(Sn, a, b) = \phi(n, a, b) + \mathrm{sg}(d(a, n) + d(b, n)).} \tag{Acc}$$

All identities leading to Acc used scalar uniqueness, coverage, and finite case analysis. No equality-reflection or induction theorem has yet been used.

E.4 Commutativity of the arithmetic maximum expression

Put

$$m(a, b) = a + d(b, a).$$

The name is descriptive only; its order-theoretic properties are not assumptions. Four zero/successor cases prove

$$m(a, b) = m(Pa, Pb) + \mathrm{sg}(a + b). \tag{Red}$$

At $(0, 0)$ both sides are zero. At $(0, Sb)$ and $(Sa, 0)$ it is the successor equation for addition. At (Sa, Sb), simultaneous subtraction cancellation gives $m(Sa, Sb) = Sa + d(b, a) = S(m(a, b))$, and the sign of the sum is one.

For fixed a, b, form the scalar terms

$$F(n) = m(d(a, n), d(b, n)) + \phi(n, a, b),$$
$$G(n) = m(d(b, n), d(a, n)) + \phi(n, a, b).$$

Red, Acc, and commutativity of addition give $F(Sn) = F(n)$ and $G(Sn) = G(n)$. Hence Const gives

$$F(n) = m(a, b), \qquad G(n) = m(b, a).$$

At $n = b$, the equality $d(b, b) = 0$ and the identities $m(c, 0) = c = m(0, c)$ make both terms equal, by these equations, to $d(a, b) + \phi(b, a, b)$. It follows that

$$\boxed{a + d(b, a) = b + d(a, b).} \tag{Max}$$

This is the common-accumulator argument for maximum commutativity. The scalar-specific ingredient is the explicit accumulator in (2) and its derivation through Boolean DD; no prior availability of arbitrary primitive recursion is required.

E.5 Reflection and recovery of equation induction

Define

$$\Delta(a, b) = \mathrm{sg}(d(a, b) + d(b, a)).$$

Coverage and separation imply $\mathrm{sg}(u) = 0 \leftrightarrow u = 0$. They also imply $u + v = 0 \leftrightarrow (u = 0 \wedge v = 0)$: a successor in either summand would make the sum a successor, contradicting separation. Thus $\Delta(a, b) = 0$ forces $d(a, b) = 0 = d(b, a)$. Max then yields

$$a = a + d(b, a) = b + d(a, b) = b.$$

Conversely $a = b$ gives both subtractions zero. Therefore

$$\boxed{\Delta(a, b) = 0 \iff a = b, \qquad \Delta(a, b) = 0 \ \vee \ \Delta(a, b) = 1.} \tag{EqRef}$$

The equality on the right of EqRef is the logic's actual equality on the whole numerical domain. Nothing restricts a, b to standard numerals or to the image of an encoding.

Finally, the independently proved UR retraction lemma gives contextual induction for all zero equations $q(n) = 0$. Given an induction premise for any equation $s(n) = t(n)$, use EqRef to transfer its base and step to the zero equation $\Delta(s(n), t(n)) = 0$, apply that zero-induction rule, and transfer the conclusion back. This recovers contextual literal-equation induction in arbitrary retained first-order contexts. The reverse derivation of UR from literal-equation induction is immediate by induction on $f(n) = I(n, f(0); h)$. Consequently scalar UR and contextual equation induction are equivalent over the common scalar computation, coverage, and separation background, once the preceding UR predecessor bootstrap is included.

F Equivalence, bi-interpretation, and the native calculus

F.1 The two proof transformations

The preceding sections explicitly derive EI inside T_{UR}. For any requested instance $s(n) = r(n)$, replace it by $\Delta(s(n), r(n)) = 0$ using EqRef, apply ZI to the transferred base and step, and use EqRef again at the target. The context Γ has remained fixed throughout. Thus this argument applies to contextual induction, not only to closed unconditional theorems.

Conversely, in T_{eq} assume the premise of UR and apply EI to

$$f(n) = I(n, f(0); u.h(u)).$$

The base is the zero computation and symmetry. At a successor, the retained recurrence premise, congruence applied to the induction equation, and successor computation give

$$f(Sn) = h(f(n)) = h(I(n, f(0); h)) = I(Sn, f(0); h).$$

This derives UR. Replacing each use of one rule by its displayed finite derivation proves equality of all derivable sequents. The transformation requires only finite manipulation of fixed terms and proofs.

F.2 What is inherited in the comparison with iPRA

The supplied project already proves that this ordinary scalar T_{eq} is bi-interpretable with full first-order iPRA. The pinned reference files are `inherited_ordinary_bridge.tex`, `equality_reflection.tex`, and `primitive_recursion.tex` under `Research/certified_zero/reference/`, at repository commit `11be5aabc6b07262046e2553419bc6b05271d178`. The exact point at which that bridge is used here is after EI has been derived independently from UR.

More explicitly, each fixed scalar iterator has its usual primitive-recursive interpretation in iPRA. Equation induction proves its uniqueness, hence validates UR. In the other direction, EqRef and the selector assign a Boolean term to every fixed quantifier-free formula. Truth tables turn its induction into equation induction. The inherited arithmetic construction supplies scalar pairing and projection terms with their inverse equations. An ordinary primitive recursion can then be simulated by iterating the paired state consisting of its counter and current value. This last use of pairing is legitimate because equation induction has already been recovered.

All quantifiers in these translations range over the whole numerical domain. Equality translates as actual equality. The two translations replace only fixed primitive-recursive operation descriptions by the proved corresponding terms, and the defining recurrences identify the round trips. Their numerical isomorphism is $x = y$. Since the identity translation between T_{UR} and T_{eq} preserves every sequent, composing it with the pinned scalar bridge preserves these same whole- domain and identity-composite properties. Mutual translation of a derivation of $\bot$ gives equiconsistency.

F.3 Replacing progress in the native presentation

There is also a direct native corollary. Retain the original terms and native formula constructors

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A,$$

and their ordinary readings

$$Q_y B = B(0) \to \forall y\, B(y), \qquad D_{x,y} B = \exists x\, (B(x,0) \to \forall y\, B(x,y)).$$

Keep Hyp, the two computation equations, equality transport, QE, DE, and DI by cases; replace equation-progress introduction by UR. This gives exactly the native sequents of the original calculus.

The logical reconstruction needed for this statement is independent of the deleted progress rule. The pinned `inherited_choice_basis.tex` derives numerical cases, binder congruence, the ordinary intuitionistic logical interface, and explosion using DI by cases, QE, DE, computation, and transport. With crossed one-term transport, the ordinary equality rules and equation-template transport are first recovered; transport through Q and D follows by structural induction on formulas using the case rules. These are inductions on finite syntax in the metatheory, not hidden numerical induction instances.

Thus the ordinary UR derivation above can be translated into native macros without an appeal to progress. It recovers EI and therefore the old progress rule. Conversely, old equation progress gives EI and the displayed proof of UR. This establishes the native equivalence. It does not by itself imply that a particular Python encoding is smaller; that claim requires the complete checker and its matched measurement.

F.4 Scope of the result

The theorem is a written internal derivation, not a proof-assistant formalization. Numerical checks of the displayed terms are useful implementation checks but do not verify the theorem in nonstandard or intuitionistic models. No claim of absolute minimality is made. The mathematical advance is the completion of the restricted scalar uniqueness basis using an explicit scalar accumulator, without assuming pairing, counter-aware recursion, or equality reflection along the way.

References

[1] V. Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt snapshot `6ffc49f55d355b548f7a8e1530570aee5de086b3`. `https://github.com/VladimirReshetnikov/ProveIt/tree/6ffc49f55d355b548f7a8e1530570aee5de086b3/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.

[2] ProveIt, `certified_zero/reference/inherited_choice_basis.tex`. Native rules, induction-independent macros and formula composites. Included under `reference/scalar_bridge/`.

[3] ProveIt, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, `primitive_recursion.tex`, under `Research/certified_zero/reference/`. Included under `reference/scalar_bridge/`.

[4] ProveIt, `Research/scalar_uniqueness/ur_complete.tex`, `ur_retraction.tex`, `ur_predecessor.tex`, `ur_reflection.tex`. Reproduced in Appendix A. `https://github.com/VladimirReshetnikov/ProveIt/tree/6ffc49f55d355b548f7a8e1530570aee5de086b3/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/scalar_uniqueness`.

[5] ProveIt, `Research/full_normalization/`, `Research/computation_normal/`, and Lean modules `FullNormalization`, `NativeFullNormal`. The overview identifies `Tm.full`, `Tm.full_subst_congr`, and `fnprf_iff_nprf`.

[6] R. L. Goodstein, *Recursive Number Theory*, Chapter V: Formalisations of Primitive Recursive Arithmetic. Studies in Logic and the Foundations of Mathematics 20 (1957), 104–118. `https://doi.org/10.1016/S0049-237X(08)71976-1`.

[7] W. W. Tait, *Primitive Recursive Arithmetic and Its Role in the Foundations of Arithmetic: Historical and Philosophical Reflections*. Author manuscript describing uniqueness and induction. `https://home.uchicago.edu/~wwtx/PRA2.pdf`. The search index supplied the historical passage; the direct URL returned 404 during this session. The proof here does not depend on that URL.

[8] ProveIt, `Research/deterministic_introduction/implementation/code/compact_optional.py`, Git blob `a0af7afb236b530eb30c63c5ab788151024271b7`. Included as `implementation/compact/base506.py`.

[9] ProveIt incoming archives `ipra_frontier_review_2026-11-12.zip`, `ipra_smaller_envelope.zip`, `ipra_orbit_closure.zip`. Their separate implementation refinements and frozen census are attributed in the archived provenance files.